Advisors' Advisor

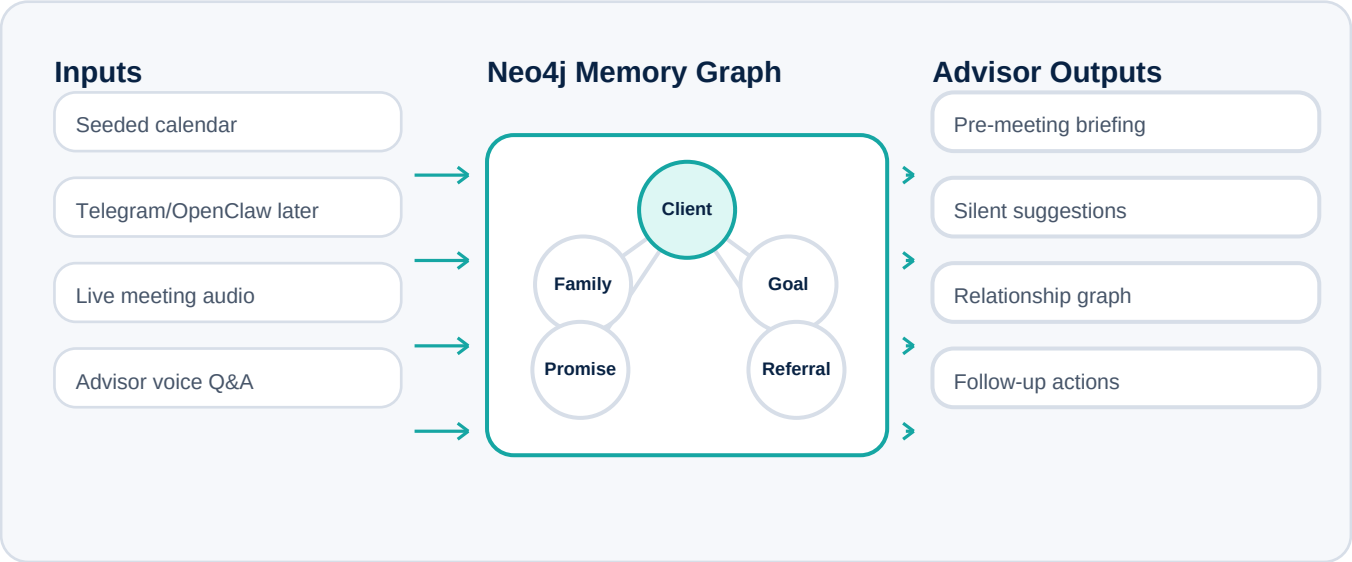
Technical PDF - architecture, data model, APIs, and implementation plan

Web-first Realtime prototype with Neo4j as the client memory graph.

Prepared as a clear, unambiguous product and technical baseline for the hackathon demo.

1. System Overview

The system is a Next.js web app that supports two primary modes: pre-meeting voice briefing and live meeting companion. Neo4j stores client memory, relationships, open loops, and referral opportunities. OpenAI Realtime powers the browser voice experience.

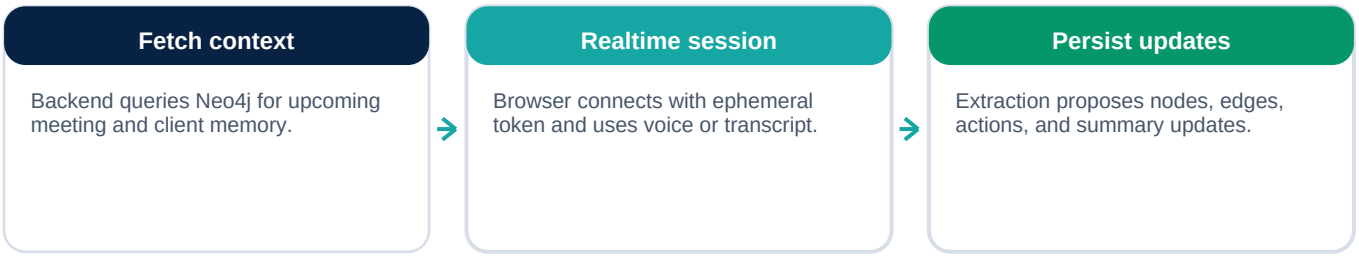


Realtime browser sessions should use a server-minted ephemeral token. The browser does not receive the standard OpenAI API key. The browser connects to the Realtime API through WebRTC and sends or receives events over the data channel.

2. Core Architecture

Layer	Recommended choice	Responsibility
Frontend	Next.js App Router + React	Calendar demo page, briefing UI, live meeting UI, graph/cards/actions.
Voice	OpenAI Realtime via WebRTC	Pre-meeting voice briefing and Q&A; live meeting transcription and analysis events.
Backend	Next.js API routes or route handlers	Mint ephemeral Realtime tokens, query Neo4j, run extraction, persist graph updates.
Memory	Neo4j	Client profile graph, events, family, referrals, goals, promises, concerns, meeting summaries.
Graph UI	React Flow or similar	Relationship graph and memory graph display.
Later chat	Telegram/OpenClaw	Advisor chat Q&A; first, passive whitelisted ingestion later.
Later calls	Vapi	Real outbound pre-meeting phone calls.

Runtime flow



3. Neo4j Data Model

Neo4j should model the advisor-client relationship explicitly so the demo can show both memory recall and referral network value. Store every extracted fact with source, confidence, and timestamp when possible.

Node label	Purpose	Example properties
Advisor	Financial advisor using the app.	id, name, firm
Client	Client profile anchor.	id, name, riskProfile, lastBriefedAt
Person	Family, friend, colleague, specialist, referral contact.	id, name, role, notes
Meeting	Scheduled or completed meeting.	id, startsAt, status, summary
LifeEvent	Important personal event.	type, description, date, confidence
Objective	Advisor goal or client goal.	type, description, status
Concern	Unresolved issue, hesitation, objection.	description, status, severity
Promise	Commitment made by advisor or client.	description, dueDate, status
Action	Post-meeting next step.	type, description, owner, status

Relationship	Meaning
(:Advisor)-[:SERVES]->(:Client)	Advisor owns the client relationship.
(:Client)-[:HAS_MEETING]->(:Meeting)	Meeting timeline for context and last-met lookup.
(:Client)-[:HAS_FAMILY_MEMBER]->(:Person)	Family context such as spouse or children.
(:Client)-[:MENTIONED]->(:Person)	Referral or network candidates.
(:Client)-[:HAS_CONCERN]->(:Concern)	Open issues and objections.
(:Meeting)-[:CREATED]->(:Promise)	Promises detected during the meeting.
(:Action)-[:RELATES_TO]->(:Client)	Follow-up tasks attached to client context.

4. MVP L1: Pre-meeting Briefing

Goal

Before a scheduled meeting, the advisor opens a simple web call UI. The assistant speaks a concise briefing and supports voice Q&A; using context fetched from Neo4j.

Implementation sequence

- Seed demo calendar with an upcoming Mr. Tan meeting.
- On briefing start, backend queries Neo4j for client profile, last meeting, open concerns, life events, referrals, and promises.
- Backend mints a Realtime ephemeral token configured for gpt-realtime-2.
- Browser creates a WebRTC session and sends a session prompt containing the fetched context.
- Assistant speaks the prepared briefing and answers advisor Q&A.;
- Frontend displays retrieved context as bullet cards and relationship graph snippets.

Acceptance criteria

- Advisor can start the briefing from the web page.
- Voice output summarizes who, last meeting, what happened, open loops, and suggested opening.
- Advisor can ask follow-up questions by voice.
- Responses stay grounded in Neo4j context and clearly say when information is unknown.

5. MVP L2: Live Meeting Companion

The live meeting companion runs silently on phone or laptop during the advisor-client conversation. It listens, extracts useful context, and displays suggestions. It does not interrupt the meeting.

Capability	Behavior
Silent suggestions	Show advisor-only suggestions such as questions, reminders, and objective gaps.
Memory retrieval	Pull relevant Neo4j facts when conversation topics match client context.
Nuance capture	Extract remarks, emotional cues, unresolved concerns, promises, and important facts.
Referral tracking	Detect mentions of family, friends, business contacts, or specialists and propose graph updates.
Post-meeting actions	Draft reminders, follow-ups, introductions, and summary notes.
Diarization	Not required for first version. Use contextual inference or manual labels only if needed.

Suggested extraction schema

Every extraction should produce structured proposals such as: type, title, description, relatedClientId, relatedPersonName, evidenceQuote, confidence, proposedNodes, proposedEdges, and recommendedAction. Low-confidence extractions should be shown for advisor review rather than auto-saved.

6. API And Data Contracts

Endpoint	Purpose
GET /api/demo/calendar	Return seeded upcoming meetings.
GET /api/clients/:id/context	Return normalized client memory from Neo4j.
POST /api/realtime/token	Mint an ephemeral Realtime token for browser WebRTC.
POST /api/meetings/:id/extract	Analyze transcript chunks and return suggestions or memory proposals.
POST /api/memory/proposals	Create pending graph update proposals.
POST /api/memory/commit	Commit approved proposals to Neo4j.
GET /api/clients/:id/graph	Return graph nodes and edges for visualization.
POST /api/actions	Create follow-up actions and reminders.

Prompt guardrails

- Do not invent facts not present in Neo4j or transcript.
- When uncertain, mark as a proposal and ask advisor to confirm.
- During live meetings, produce silent text suggestions only.
- Never address or message the client directly in L1 or L2.
- Prioritize relevance, brevity, and advisor actionability.

7. Staged Roadmap

Stage	Build target	Dependency risk
L1	Realtime web pre-meeting briefing and Q&A; over Neo4j context.	OpenAI key, mic permission, Neo4j seeded data.
L1.5	Dynamic visual display: cards, table, graph, chart, bullets.	Frontend polish and graph layout.
L2	Silent meeting companion with suggestions, memory fetch, extraction, and action generation.	Prompt quality and transcript handling.
L3	Telegram/OpenClaw advisor Q&A; over client memory.	OpenClaw setup and bot/channel config.
L3.1	Whitelisted passive chat profiling.	Consent, filtering, source attribution.
L3.2	Automated follow-up drafts and approved sends.	Outbound safety and approval workflow.
L4	Real outbound phone calls through Vapi.	Phone integration and scheduling trigger.
L5	WhatsApp ingestion via OpenClaw.	Provider access, whitelist, privacy model.

8. Demo Data: Sarah And Mr. Tan

Use one polished scenario end to end. This avoids broad demo sprawl and lets the audience understand the product in a realistic advisor workflow.

Entity	Seeded details
Advisor	Sarah Lim, financial advisor.
Client	Mr. Tan Wee Seng, long-term client, family-oriented, moderate risk profile.
Last meeting	Three months ago. Discussed policy renewal hesitation and unresolved will planning.
Life event	Daughter Jia En got into NUS.
Open concern	Will planning still unresolved.
Referral cue	Asked whether Sarah knows a lawyer; potential intro to Marcus Lee.
Specialist	Evelyn Ng, estate planner, suitable for estate planning conversation.
Meeting objective	Strengthen relationship, explore estate planning, clarify renewal hesitation, identify referral opportunities.

Final technical success criterion

The demo is technically successful when the same Neo4j memory graph powers the pre-meeting voice briefing, the on-screen live meeting suggestions, the post-meeting actions, and the visible relationship graph updates.

9. Known Open Items

- Choose exact graph visualization library after implementation starts; React Flow is a strong default.
- Decide whether L2 uses a single Realtime session for both transcription and suggestions or uses Realtime plus separate server-side extraction calls.
- Define the approval threshold for committing extracted memories to Neo4j.
- Define privacy/consent language before any production or sponsor-facing pilot.
- Confirm teammate feedback on whether Mem0 is still useful as an optional retrieval layer. Current baseline keeps Neo4j as source of truth.

These are implementation choices, not product ambiguities. The product direction and demo scope are fixed by the decisions in this document.